



中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类

中山大学计算机学院



主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



中山大学MOOC课程组

目录

CONTENTS

01

对象指针

02

=delete 和 =default

03

对象成员初始化

04

对象的内存布局

05

拷贝构造函数

06

传值和传引用

07

常引用\常方法\常量正确性



对象成员初始化

- 类的非静态数据成员可以用下列几种方式之一初始化：
 - 在构造函数的成员**初始化器**列表中。（C++11）
 - 通过**默认成员初始化器**，它是成员声明中包含的花括号或等号**初始化器**。（C++11）
 - 构造函数体内进行赋值操作。（注意：**不能构造成员**，除非你特别熟悉 `new` 的各种用法）
- 为什么需要初始化器列表和默认成员初始化器？
 - 对象初始化分两个阶段：首先按**声明顺序**初始化成员、**然后执行构造函数函数体**；
 - 对于复杂对象成员，如 `std::string name`，必须先构造才能在构造函数中赋值。这会导致构造函数必须先构造一个 `string` 中间变量才能赋值给 `name`；
 - 对于引用类型成员，`const` 成员需要预初始化；
 - 对象成员**初始化需要参数**。
- 为了提升初始化性能，C++引入了默认成员初始化器和初始化器列表



成员初始化器列表 (C++11) - 语法

与其他函数不同，构造函数除了有名字，参数列表和函数体之外，还可以有初始化器列表，初始化器列表以冒号开头，后跟一系列以逗号分隔的成员初始化器。

```
class DATE {  
public:  
    DATE( int initYear, int initMonth, int initDay )  
        :year(initYear), month(initMonth),  
day(initDay){};  
    DATE();  
private:  
    int month;  
    int day;  
    int year;  
};
```



成员初始化器列表 - 要点

必须使用初始化器列表的时候

除了性能问题之外，有些时候合初始化器列表是不可或缺的，以下几种情况时必须使用初始化器列表：

1. **常量成员**，因为常量只能初始化不能赋值，所以必须放在初始化器列表里面
2. 引用类型，**引用必须在定义的时候初始化**，并且不能重新赋值，所以也要使用初始化器
3. 没有默认构造函数的类类型，因为使用初始化器列表可以不必调用无参构造函数来初始化，而是直接其他构造函数初始化

成员初始化器列表 - 要点

成员变量声明的顺序

成员是按照他们在类中声明的顺序进行初始化的，而不是按照他们在初始化器列表出现的顺序初始化的：

```
class foo {  
public:  
    int i ;int j ;  
    foo(int x):i(x), j(i){}; // ok, 先初始化i, 后初始化j  
};
```



成员初始化器列表 - 要点

成员变量声明的顺序

再看下面的代码：

```
class foo
{
public:
    int i ;int j ;
    foo(int x):j(x), i(j){} // i值未定义
};
```

这里*i*的值是未定义的因为虽然*j*在初始化器列表里面出现在*i*前面，但是*i*先于*j*定义，所以先初始化*i*，而*i*由*j*初始化，此时*j*尚未初始化，所以导致*i*的值未定义。一个好的习惯是，**按照成员声明的顺序进行初始化**。



对象成员初始化 - 例子

```
/*member initialization*/
struct S {
    int n;                // 非静态数据成员
    int& r = n;           // 引用类型的非静态数据成员; =默认构造
    int a[2] = {1, 2};    // 带默认成员初始化的非静态数据成员 (C++11)
    std::string s{'H','C'}; // 带默认成员初始化器
    struct NestedS {
        std::string s;
        NestedS(std::string s = "hello"):s(s) {}; //构造函数
    } d5;                // 具有嵌套类型的非静态数据成员

    const char bit : 2;    // 2 位的位域, const 初始化

    S():n(7),bit(3) {}     // ":n(7),bit(3)" 是初始化器列表; "{}" 是函数体
    S(int x):n{x},bit(3) {}
};

int main()
{
    S s;                // 调用 S::S()
    S s2(10);           // 调用 S::S(int)
}
```




对象成员初始化 - 练习

- 修改 `S()`，请输出成员 `r` 的值，请解释结果
- 修改成员 `d5` 的构造函数输出嵌套成员 `s` 的值，请问先输出 `r` 还是 `d5.s` ?
- 修改 `S()`，请输出成员 `s` 的值，请解释结果
- 请修改成员 `s` 的初始值，使用 `std::string(4, 'C')` 初始化 `s`
- 请修改 `d5.s` 的初始值为 "world"，请给出至少两种初始化 `d5` 的方法
- 修改 `S()` 的初始化列表，删除 `bit(3)`，请解释结果
 - 在构造函数内给 `bit` 赋值，可行吗？
 - 在声明中给出默认初始化器，可行吗？



对象的内存布局

C++程序的内存格局通常分为四个区：

全局数据区(data area)

代码区(code area)

栈区(stack area)

堆区(heap area)(即自由存储区)。

全局数据区存放全局变量，静态数据和常量。

所有类成员函数和非成员函数代码存放在代码区；

为运行函数而分配的局部变量、函数参数、返回数据、返回地址等存放在栈区；

余下的空间都被称为堆区。



对象的内存布局

在类的定义时，类的成员函数被放在代码区。

类的静态成员变量在全局数据区。

非静态成员变量在类的实例内，实例在栈区或者堆区。

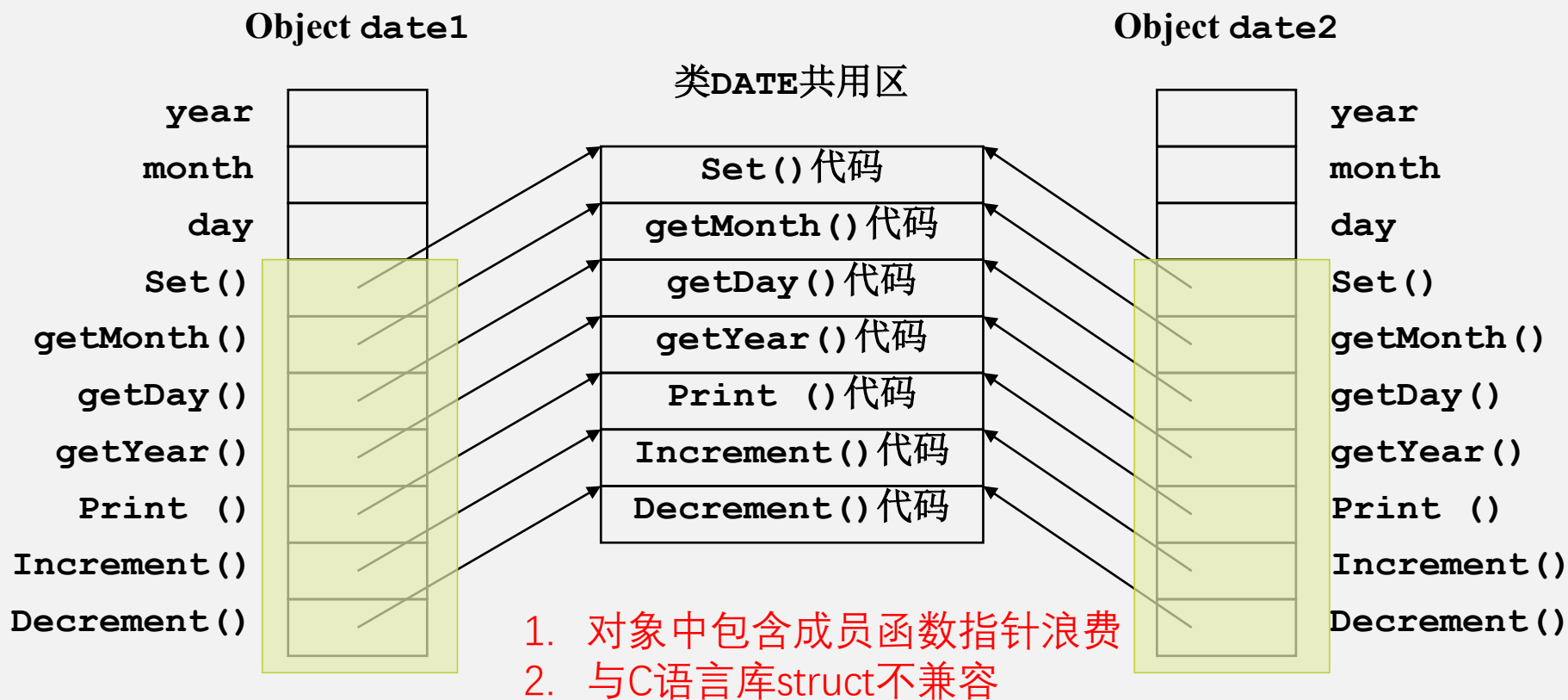
虚函数指针、虚基类指针在类的实例内，实例在栈区或者堆区。

类的实例如果是定义的类型变量，则在栈内存区，

如果是new出来的类指针，则在堆内存区，同时引用会保存在栈里。



对象的内存布局?



C++对象的内存布局与静态联编技术

```
DATE date1();  
DATE *date2 = new DATE();  
  
date1.Increment();  
date2->print();
```

C++如何翻译这段代码呢？

1. 在栈上按非静态数据结构分配空间，并调用构造函数初始化
2. 在堆上做类似操作
3. 将 `date1.Increment(...)` 翻译为 `DATE::Increment(&date1, ...)`
4. 类似 `DATE::Print(date2, ...)`

对象方法静态联编：指在编译阶段，就能直接使用代码段函数地址调用动态对象的方法。该方法仅需要向非静态成员函数传送this指针，即可用静态函数调用实现动态调用效果。

DATE date1

year	
month	
day	

DATE date2

year	
month	
day	

类DATE的代码段

Set () 代码
getMonth () 代码
getDay () 代码
getYear () 代码
Print () 代码
Increment () 代码
Decrement () 代码

静态联编的优势：

1. 对象布局与C结构内存布局一致，使得内存中对象便于与其他语言程序库兼容
2. 高效率，高性能

拷贝构造函数 - 概念

拷贝构造函数：在定义语句中用同类型的对象初始化另一个对象。

//假定**C**为已定义的类，则

C obja; //调用**C**的 **(1) 无参构造函数**，如无**(1)(2)**构造函数则用**默认构造函数**

C obja(1,2) //调用**C**的有参 **(2) 普通构造函数**

/*调用C**的 **(3) 拷贝构造函数**用对象**obja**初始化对象**objb**。如果有为**C**类明确定义拷贝构造函数，将调用这个拷贝构造函数；如果没有为**C**类定义拷贝构造函数，将调用默认拷贝构造函数。*/**

C objb(obja); //或

C objb = obja ; //两者等价 **(注意：不是赋值运算)**



拷贝构造函数 - 语法

用**类类型本身**作形式参数。

该参数传递方式为**按引用传递**，避免在函数调用过程中生成形参副本。

该形参声明为**const**，以确保在拷贝构造函数中不修改实参的值

C::C(const C& obj);

例如： **COMPLEX(const COMPLEX& other);**

注： C::C(C& obj) 不用 const 形式已过时。

C::C(C&& obj) 形式称为移动构造函数（超纲）



拷贝构造函数：要点

形参类型为该类类型本身且参数传递方式为按引用传递。

用一个已存在的该类对象初始化新创建的对象。

每个类都**必须有**拷贝构造函数：

用户可根据自己的需要显式定义拷贝构造函数。

若用户未提供，则该类使用由系统提供的**缺省拷贝构造函数（可用=default）**，也可用 **=delete** 弃置该函数。

缺省拷贝构造函数**按照初始化顺序**，对对象的各基类和**非静态成员进行完整的逐成员复制**，完成新对象的初始化。即逐一调用成员的拷贝构造，如果成员是基础类型，则复制值（赋值）。

隐式调用复制构造 (1) : 对象作为函数值参

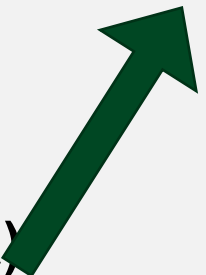
除了用 声明 `C x(c)` 或 `new C(c)` 会触发复制构造外，
将一个对象作为实参，以按值调用方式传递给被调函数的形参对象。

假定 **C** 为已定义的类，**obja** 为 **C** 类对象，且

```
void fun(C temp)
{
    ...
}
```

则

`fun(obja)`



用 **obja** 来初始化 **temp**。如果有为 **C** 类明确定义拷贝构造函数，将调用这个拷贝构造函数；如果没有为 **C** 类定义拷贝构造函数，将调用缺省的拷贝构造函数。

// **obja** 传递给 **fun** 函数，创建形参对象 **temp** 时，调用 **C** 的拷贝构造函数用对象 **obja** 初始化对象 **temp**，**temp** 生存期结束被撤销时，调用析构函数



隐式调用复制构造 (2) : 对象作为值从函数返回

生成一个临时对象作为函数的返回结果：当函数返回一对象时，系统将自动创建一个临时对象来保存函数的返回值。创建此临时对象时调用拷贝构造函数，当函数调用表达式结束后，撤销该临时对象时，调用析构函数。

```
C fun2()           //假定C为已定义的类，且
{
    C t;
    ...
    return t ;
}
则：
b = fun2( );
```

t	-->	临时对象	-->	b
拷贝构造函数				赋值运算符



对象作为值返回的编译优化

```
C func()  
{  
    C tmp;  
    ...  
    return tmp;  
}
```

但现在的gcc/g++不这么处理，会做一个优化。在C函数里有个tmp变量，离开func时不撤销这个对象，而是让new C和这个对象关联起来。也就是说tmp的地址和new C是一样的。



例子

```
//例子: code11
class F00 {
public:
    F00(int i)
    { cout << "Constructing.\n";
      member = i;
    }
    F00(const F00& other)
    { cout << "Copy constructing.\n";
      member = other.member;
    }
    ~F00()
    { cout << "Destructing.\n"; }
    int get()
    { return member; }

private:
    int member;
};
```

```
void display ( F00 obj )
{
    cout << obj.get() << "\n";
}

F00 get_foo( )
{
    int value;

    cout<< "Input an integer:";
    cin>>value;
    F00 obj(value);

    return obj;
}
```



例子

```
int main()
{
    F00 obj1(15);          // 调用一般的构造函数

    F00 obj2 = obj1;       // 调用拷贝构造函数

    display(obj2);         // 调用拷贝构造函数用实参obj2初始化形参obj

    obj2=get_foo(); // 调用拷贝构造函数创建作为返回结果的临时对象

    display(obj2);

    return 0;
}
```



例子

运行结果分析

Constructing.

Copy constructing.

Copy constructing.

15

Destructing.

Input an integer: 7

Constructing.

Copy constructing.

Destructing.

Destructing.

Copy constructing.

7

Destructing.

Destructing.

Destructing.

创建obj1

创建obj2, 用obj1对其进行初始化

创建obj, obj2初始化obj(display里面)

输出obj的值 (display函数里)

撤销obj (执行离开display函数时)

输入7

创建obj (get_foo函数里)

创建作为返回值的临时对象

撤销temp_obj

撤销作为返回值的临时对象

创建obj, obj2初始化obj(display里面)

输出obj的值

撤销obj

撤销obj2

撤销obj1



复制策略：拷贝构造函数自定义

1. 对于不含指针成员类，使用系统提供（编译器合成）的**默认拷贝构造函数**即可。
2. 缺省拷贝构造函数使用**浅复制策略**，不能满足对**含指针数据成员**的类需要。
3. 含指针成员类通常应重写以下内容：
 - **构造函数**（及拷贝构造函数）中分配内存，**深复制策略**
 - **= 操作**重写，完成对象**深复制策略**
 - **析构函数**中释放内存

浅拷贝只复制成员指针的值，而不复制指向的对象实体，导致新旧对象成员指针指向同一块内存。但深拷贝要求成员指针指向的对象也要复制，新对象跟原对象的成员指针不会指向同一块内存，修改新对象不会改到原对象。



例子

```
//例子: code12
class NAME {
public:
    NAME();
    ~NAME();
    void show();
    void set(char* s);
private:
    char* str;
};

NAME::NAME() {
    str = NULL;
    cout << "Constructing.\n";
}

NAME::~~NAME() {
    cout << "Destructing.\n";
    if (str != NULL)
        delete []str;
}
```

```
void NAME::show()
{
    cout << str << "\n";
}

void NAME::set(char* s)
{
    if (str!=NULL)
        delete []str;

    str = new char[strlen(s) + 1];

    if (str!=NULL)
        strcpy(str, s);
}
```




分析

```
NAME get_name()
{
    NAME obj;    //①
    char temp_str[250]; //②
    cout << "Input your name: ";
    cin >> temp_str;
    obj.set(temp_str); //③
    return obj; //④
} //⑤
```

```
int main()
{
    NAME myname;
    myname = get_name();
    myname.show();
}
```

1. 创建myname

myname.str **NULL**

2. 调用get_name

①创建局部对象obj，**注意：它就是函数返回对象**

obj.str **NULL**

②读入temp_str

L e e \0

③调用set函数，new、复制2000H

obj.str **2000H** → **L e e \0**

④ get_name函数返回，创建临时对象，调用缺省的拷贝构造函数，**注意：优化后无此步骤**

临时对象.str **2000H**



分析

- ⑤撤销局部变量obj：由new运算分配的空间（地址为2000H）通过delete运算被释放。
- 3. 临时对象赋给myname，调用系统提供的缺省赋值运算符，实施浅复制，从而myname.str的值置为2000H。**注意：优化后为返回（栈顶）对象赋值给 myname**
- 4. 撤销临时对象，释放其str成员指向的内存空间（地址为2000H）。因为该内存空间已释放，再次释放会出现问题。**注意：优化后为释放栈顶对象（obj），释放空间**
- 5. myname.show()输出对象myname的str成员指向的内存空间（地址为2000H）中的内容，该内容不确定。
- 6. 撤销对象myname，释放其str成员指向的内存空间（地址为2000H）。Delete运算再次释放产生内存错误。



分析

运行结果：

Constructing.

创建myname

Constructing.

创建obj

Input your name: Lee↵

输入Lee

Destructing.

撤销obj, delete str

Destructing.

撤销函数返回的临时对象

Null pointer assignment

delete出问题

××× （此行输出的字符串是不确定的！）

Destructing.

撤销myname

Null pointer assignment

delete出问题

改进

修改：加入拷贝构造函数的定义

```
NAME::NAME(NAME& other)
```

```
{
```

```
    cout << "Copy Constructing.\n";
```

```
    if (other.str == NULL)
```

```
    {    str=NULL;
```

```
    return;
```

```
    }
```

```
    str=new char[strlen(other.str)+1];
```

```
    if (str!=NULL)
```

```
    strcpy(str, other.str);
```

```
    //code13
```

④ get_name函数返回，调用拷贝构造函数创建临时对象：

obj.str

2000H → L e e \0

临时对象.str

3000H → L e e \0

```
void main()
```

```
{
```

```
    NAME myname;
```

```
    myname = get_name();
```

```
    myname.show();
```

```
}
```



改进

修改：加入赋值运算符重载函数的定义

```
NAME& operator=(const NAME& other)
```

```
{
```

```
    if (other.str == NULL)
```

```
    {str=NULL;
```

```
        return *this;
```

```
    }
```

```
    if (str!=NULL)    delete []str;
```

```
    str=new char[strlen(other.str)+1];
```

```
    if (str!=NULL)    strcpy(str, other.str);
```

```
    return *this; /*若函数的返回值类型为NAME，则此语句会引起
```

对拷贝构造函数的调用；若函数的返回值类型为

NAME&，则不调用拷贝构造函数*/

```
}
```

```
void main()
```

```
{
```

```
    NAME myname;
```

```
    myname = get_name();
```

```
    myname.show();
```

```
}
```



拷贝构造函数

1. 对于不含指针成员类，使用系统提供（编译器合成）的缺省拷贝构造函数即可。
2. 缺省拷贝构造函数使用浅复制策略，因此对含指针成员类并不能满足需要。
3. 含指针成员类通常应在构造函数（及拷贝构造函数）中分配内存，在析构函数中释放内存。

浅拷贝只复制指向某个对象的指针，而不复制对象本身，新旧对象还是共享同一块内存。但深拷贝会另外创建一个一模一样的对象，新对象跟原对象不共享内存，修改新对象不会改到原对象。



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



主讲人：

